FILE Engine

1. Kompakte System-Referenz für KI-Modelle
2. Operationen
3. EXEC – Dateioperationen
4. READ – Datei lesen
    4.1 Parameter
5. WRITE – Datei schreiben
    5.1 Parameter
6. APPEND – Text anhängen
    6.1 Parameter
7. COPY – Datei kopieren
    7.1 Parameter
8. MOVE – Datei verschieben
    8.1 Parameter
9. DELETE – Datei löschen
    9.1 Parameter
10. EXISTS – Existenz prüfen und speichern
11. Relative und absolute Pfade
12. FILE Engine und OC Engine
13. BUFFER – Datenextraktion
14. BUFFER Actions
    14.1 TEXT
    14.2 FILTER
    14.3 LINE
    14.4 LAST_LINE
    14.5 COUNT
15. ASSERT – Validierung
16. Filesystem Assertions
    EXISTS
    NOT_EXISTS
17. Content Assertions
    17.1 CONTAINS
    17.2 NOT_CONTAINS
    17.3 EQUALS
    17.4 NOT_EQUALS
    17.5 COUNT
18. EXISTS: EXEC vs. ASSERT
    EXEC EXISTS
    ASSERT EXISTS
19. Platzhalter
20. Vollständiges Beispiel
21. Troubleshooting & Best Practices
    Datei wird nicht gefunden
    Relative Pfade
    Cleanup
    BUFFER benötigt READ
    Content ASSERT benötigt response
22. Kurzreferenz
23. Design-Grundsatz

Die **FILE Engine** ermöglicht Dateioperationen auf dem lokalen Dateisystem des AGATE-Prozesses.

Sie unterstützt:

- Lesen von Dateien
- Schreiben und Überschreiben von Dateien
- Anhängen von Text
- Kopieren und Verschieben von Dateien
- Löschen von Dateien
- Prüfen der Dateiexistenz
- Extraktion von Dateiinhalten über `BUFFER`
- Validierung von Dateiinhalten und Dateiexistenz über `ASSERT`

Die FILE Engine arbeitet ausschließlich auf dem **lokalen Dateisystem des AGATE-Prozesses**.

---

1. Kompakte System-Referenz für KI-Modelle

```
# AGATE FILE-Engine Syntax Referenz
# Nutze dies für Testfall-Generierung und Analyse

# READ
- type: FILE
  op: EXEC
  action: READ
  path: "..."
  response: "..."
  encoding: "UTF-8"          # Optional, Default: UTF-8
  condition: "..."           # Optional

# WRITE
- type: FILE
  op: EXEC
  action: WRITE
  path: "..."
  text: "..."
  encoding: "UTF-8"          # Optional, Default: UTF-8
  overwrite: true            # Optional
  condition: "..."

# APPEND
- type: FILE
  op: EXEC
  action: APPEND
  path: "..."
  text: "..."
  encoding: "UTF-8"          # Optional, Default: UTF-8
  newline: false             # Optional, Default: false
  condition: "..."

# COPY
- type: FILE
  op: EXEC
  action: COPY
  source: "..."
  target: "..."
  overwrite: false           # Optional
  condition: "..."

# MOVE
- type: FILE
  op: EXEC
```

```
  action: MOVE
  source: "..."
  target: "..."
  overwrite: false           # Optional
  condition: "..."

# DELETE
- type: FILE
  op: EXEC
  action: DELETE
  path: "..."
  missingOk: false           # Optional, Default: false
  condition: "..."

# EXISTS
- type: FILE
  op: EXEC
  action: EXISTS
  path: "..."
  response: "..."
  condition: "..."

# BUFFER
- type: FILE
  op: BUFFER
  response: "..."
  action: <TEXT|FILTER|LINE|LAST_LINE|COUNT>
  value: "..."
  name: "..."
  condition: "..."

# ASSERT
- type: FILE
  op: ASSERT
  action:
<EXISTS|NOT_EXISTS|CONTAINS|NOT_CONTAINS|EQUALS|NOT_EQUALS|COUNT>
  path: "..."
  response: "..."
  value: "..."
  expected: "..."
  condition: "..."

```

---

2. Operationen

Die FILE Engine verwendet dieselbe Grundstruktur wie andere AGATE Engines:

```
type   = Engine
op     = Ausführungsmodus
action = konkrete Dateioperation

```

Beispiel:

```
- type: FILE
  op: EXEC
  action: READ

```

Die Engine unterstützt drei `op`-Varianten:

Operation	Beschreibung
`EXEC`	Führt eine Dateioperation aus.

`BUFFER`	Extrahiert Informationen aus einem zuvor gelesenen Dateiinhalt.
`ASSERT`	Validiert Dateiexistenz oder Dateiinhalt.

---

3. EXEC – Dateioperationen

`EXEC` führt eine konkrete Dateioperation aus.

Unterstützte Actions:

```
READ
WRITE
APPEND
COPY
MOVE
DELETE
EXISTS

```

---

4. READ – Datei lesen

`READ` liest den vollständigen Inhalt einer Datei und speichert ihn unter `response`.

```
- type: FILE
  op: EXEC
  action: READ
  path: "output/report.txt"
  response: report

```

Der gespeicherte Inhalt kann anschließend mit `BUFFER` oder `ASSERT` weiterverarbeitet werden.

4.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
`path`	Pfad der zu lesenden Datei	Ja	–
`response`	Name für den gelesenen Dateiinhalt	Ja	–
`encoding`	Zeichencodierung	Nein	`UTF-8`

`condition`	Bedingung für die Ausführung	Nein	–

Beispiel mit explizitem Encoding:

```
- type: FILE
  op: EXEC
  action: READ
  path: "output/report.txt"
  response: report
  encoding: UTF-8

```

---

5. WRITE – Datei schreiben

`WRITE` schreibt Text in eine Datei.

```
- type: FILE
  op: EXEC
  action: WRITE
  path: "output/report.txt"
  text: "AGATE FILE Engine"

```

Mehrzeilige Inhalte können direkt über YAML angegeben werden:

```
- type: FILE
  op: EXEC
  action: WRITE
  path: "output/report.txt"
  text: |
    AGATE FILE Engine
    Line 2
    Line 3

```

5.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
`path`	Zielpfad	Ja	–
`text`	Zu schreibender Inhalt	Ja	–
`encoding`	Zeichencodierung	Nein	`UTF-8`
`overwrite`	Vorhandene Datei überschreiben	Nein	Engine-Default
`condition`	Bedingung für die Ausführung	Nein	–

Beispiel:

```
- type: FILE
  op: EXEC
  action: WRITE
  path: "output/report.txt"
  text: "{B[report_content]}"
  overwrite: true

```

---

6. APPEND – Text anhängen

`APPEND` fügt Text an eine bestehende Datei an.

```
- type: FILE
  op: EXEC
  action: APPEND
  path: "output/report.txt"
  text: "additional line"

```

Standardmäßig wird kein zusätzlicher Zeilenumbruch eingefügt.

```
newline = false

```

Soll nach dem angehängten Text ein Zeilenumbruch erzeugt werden:

```
- type: FILE
  op: EXEC
  action: APPEND
  path: "output/report.txt"
  text: "additional line"
  newline: true

```

6.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
`path`	Zieldatei	Ja	–
`text`	Anzuhängender Text	Ja	–
`encoding`	Zeichencodierung	Nein	`UTF-8`
`newline`	Zeilenumbruch anhängen	Nein	`false`
`condition`	Bedingung für die Ausführung	Nein	–

---

7. COPY – Datei kopieren

`COPY` kopiert eine Datei von `source` nach `target`.

```
- type: FILE
  op: EXEC
  action: COPY
  source: "output/report.txt"
  target: "backup/report.txt"

```

Soll eine vorhandene Zieldatei ersetzt werden:

```
- type: FILE
  op: EXEC
  action: COPY
  source: "output/report.txt"
  target: "backup/report.txt"
  overwrite: true

```

7.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
`source`	Quelldatei	Ja	–
`target`	Zieldatei	Ja	–
`overwrite`	Vorhandene Zieldatei überschreiben	Nein	`false`
`condition`	Bedingung für die Ausführung	Nein	–

---

8. MOVE – Datei verschieben

`MOVE` verschiebt eine Datei von `source` nach `target`.

```
- type: FILE
  op: EXEC
  action: MOVE
  source: "output/report.txt"
  target: "archive/report.txt"

```

`MOVE` kann auch zum Umbenennen verwendet werden.

Beispiel:

```
- type: FILE
  op: EXEC
  action: MOVE
  source: "output/report.txt"
  target: "output/report-old.txt"

```

Dadurch ist keine separate `RENAME`-Action erforderlich.

8.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
`source`	Quelldatei	Ja	–
`target`	Zielpfad	Ja	–
`overwrite`	Vorhandenes Ziel überschreiben	Nein	`false`
`condition`	Bedingung für die Ausführung	Nein	–

---

9. DELETE – Datei löschen

`DELETE` löscht eine Datei.

```
- type: FILE
  op: EXEC
  action: DELETE
  path: "output/report.txt"

```

Standardmäßig schlägt der Schritt fehl, wenn die Datei nicht existiert.

```
missingOk = false

```

Falls eine nicht vorhandene Datei akzeptiert werden soll:

```
- type: FILE
  op: EXEC
  action: DELETE
  path: "output/report.txt"
  missingOk: true

```

Dies ist besonders für Cleanup-Schritte sinnvoll.

9.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
`path`	Zu löschende Datei	Ja	–

`missingOk`	Fehlende Datei als Erfolg behandeln	Nein	`false`
`condition`	Bedingung für die Ausführung	Nein	–

---

10. EXISTS – Existenz prüfen und speichern

`EXISTS` prüft, ob eine Datei existiert, und speichert das Ergebnis als Boolean unter `response`.

```
- type: FILE
  op: EXEC
  action: EXISTS
  path: "output/report.txt"
  response: report_exists

```

Anschließend kann das Ergebnis verwendet werden:

```
{B[report_exists]}

```

Beispielsweise in einer Condition:

```
- type: FILE
  condition: "'{B[report_exists]}' == 'true'"
  op: EXEC
  action: READ
  path: "output/report.txt"
  response: report

```

Wichtig:

```
FILE EXEC / EXISTS

```

erzeugt einen Wert für die weitere Verarbeitung.

Das ist etwas anderes als:

```
FILE ASSERT / EXISTS

```

welches direkt eine Prüfung durchführt.

---

11. Relative und absolute Pfade

Die FILE Engine arbeitet auf dem lokalen Dateisystem des AGATE-Prozesses.

Absolute Pfade werden unverändert verwendet.

Beispiel:

```
path: "C:\\tmp\\report.txt"

```

Relative Pfade werden relativ zum Arbeitsverzeichnis des AGATE-Prozesses aufgelöst.

Beispiel:

```
path: "output/report.txt"

```

Wenn AGATE beispielsweise aus folgendem Verzeichnis gestartet wurde:

```
C:\work\projects\agate-server

```

entspricht der relative Pfad:

```
output/report.txt

```

dem effektiven Pfad:

```
C:\work\projects\agate-server\output\report.txt

```

Dasselbe gilt für:

```
path
source
target

```

---

12. FILE Engine und OC Engine

Die FILE Engine arbeitet immer lokal.

```
FILE
    -> lokales Dateisystem des AGATE-Prozesses

```

Dateien innerhalb eines OpenShift-Pods werden über die OC Engine verarbeitet.

Beispiel:

```
FILE READ
    -> lokale Datei lesen

OC EXEC cat ...
    -> Datei im Pod lesen

OC GET
    -> Datei aus Pod nach lokal kopieren

OC PUT
    -> lokale Datei in Pod kopieren

```

Damit bleibt die Verantwortlichkeit klar getrennt.

---

13. BUFFER – Datenextraktion

`BUFFER` arbeitet auf dem Inhalt eines vorherigen `FILE EXEC / READ`.

Beispiel:

```
- type: FILE
  op: EXEC
  action: READ
  path: "output/report.txt"
  response: report_raw

- type: FILE
  op: BUFFER
  response: report_raw
  action: TEXT
  name: report

```

Wichtig:

`BUFFER` liest nicht direkt erneut von einem Dateipfad.

Es arbeitet auf dem unter `response` gespeicherten Dateiinhalt.

---

14. BUFFER Actions

Unterstützte Actions:

```
TEXT
FILTER
LINE
LAST_LINE
COUNT

```

Action	Beschreibung	Parameter
`TEXT`	Speichert den kompletten getrimmten Dateiinhalt	–
`FILTER`	Speichert Zeilen, die einen Suchbegriff enthalten	`value`
`LINE`	Extrahiert eine Zeile; `0` = erste Zeile	`value` optional
`LAST_LINE`	Extrahiert eine Zeile relativ zum Dateiende	`value` optional
`COUNT`	Zählt Vorkommen eines Suchbegriffs	`value`

14.1 TEXT

```
- type: FILE
  op: BUFFER
  response: report_raw
  action: TEXT
  name: report

```

14.2 FILTER

```
- type: FILE
  op: BUFFER
  response: report_raw
  action: FILTER
  value: "ERROR"
  name: error_lines

```

Es werden nur Zeilen gespeichert, die den Suchbegriff enthalten.

14.3 LINE

```
- type: FILE
  op: BUFFER
  response: report_raw
  action: LINE
  value: 0
  name: first_line

```

Dabei gilt:

```
0 = erste Zeile
1 = zweite Zeile
2 = dritte Zeile
...

```

Wird `value` nicht angegeben, wird `0` verwendet.

14.4 LAST_LINE

```
- type: FILE
  op: BUFFER
  response: report_raw
  action: LAST_LINE
  value: 0
  name: last_line

```

Dabei gilt:

```
0 = letzte Zeile
1 = vorletzte Zeile
2 = drittletzte Zeile
...

```

14.5 COUNT

```
- type: FILE
  op: BUFFER
  response: report_raw
  action: COUNT
  value: "CHANGED"
  name: changed_count

```

Das Ergebnis steht anschließend über:

```
{B[changed_count]}

```

zur Verfügung.

---

15. ASSERT – Validierung

Die FILE Engine unterstützt zwei Arten von Assertions:

```
1. Filesystem Assertions
2. Content Assertions

```

---

16. Filesystem Assertions

Filesystem Assertions arbeiten direkt mit einem Pfad.

Unterstützte Actions:

```
EXISTS
NOT_EXISTS

```

EXISTS

```
- type: FILE
  op: ASSERT
  action: EXISTS
  path: "output/report.txt"

```

Der Schritt ist erfolgreich, wenn die Datei existiert.

NOT_EXISTS

```
- type: FILE
  op: ASSERT
  action: NOT_EXISTS
  path: "output/report.txt"

```

Der Schritt ist erfolgreich, wenn die Datei nicht existiert.

---

17. Content Assertions

Content Assertions arbeiten auf dem `response` eines vorherigen `READ`.

Unterstützte Actions:

```
CONTAINS
NOT_CONTAINS
EQUALS
NOT_EQUALS
COUNT

```

17.1 CONTAINS

```
- type: FILE
  op: ASSERT
  response: report_raw
  action: CONTAINS
  value: "CHANGED"

```

Prüft, ob der Suchtext im Dateiinhalt vorkommt.

17.2 NOT_CONTAINS

```
- type: FILE
  op: ASSERT
  response: report_raw
  action: NOT_CONTAINS
  value: "ERROR"

```

Prüft, ob der Suchtext nicht vorkommt.

17.3 EQUALS

```
- type: FILE
  op: ASSERT
  response: report_raw
  action: EQUALS
  value: "AGATE FILE Engine"

```

Prüft auf exakte Übereinstimmung nach der für die Engine vorgesehenen Textnormalisierung.

17.4 NOT_EQUALS

```
- type: FILE
  op: ASSERT
  response: report_raw
  action: NOT_EQUALS
  value: "unexpected value"

```

Prüft, dass der Inhalt nicht dem Vergleichswert entspricht.

17.5 COUNT

```
- type: FILE
  op: ASSERT
  response: report_raw
  action: COUNT
  value: "CHANGED"
```

```
  expected: 2

```

Prüft die Anzahl der Vorkommen eines Suchbegriffs.

---

18. EXISTS: EXEC vs. ASSERT

Die FILE Engine unterstützt `EXISTS` bewusst in zwei Varianten.

EXEC EXISTS

```
- type: FILE
  op: EXEC
  action: EXISTS
  path: "output/report.txt"
  response: exists_result

```

Ergebnis:

```
true / false

```

Das Ergebnis kann später verwendet werden.

ASSERT EXISTS

```
- type: FILE
  op: ASSERT
  action: EXISTS
  path: "output/report.txt"

```

Der Testschritt schlägt direkt fehl, wenn die Datei nicht existiert.

Kurz gesagt:

```
EXEC EXISTS
    -> liefert Information

ASSERT EXISTS
    -> validiert eine Erwartung

```

---

19. Platzhalter

Dateipfade und Inhalte können normale AGATE-Platzhalter verwenden.

Beispiel:

```
variables:
  baseDir: "output"
  filename: "report.txt"

steps:

  - type: FILE
    op: EXEC
    action: WRITE
    path: "{B[baseDir]}/{B[filename]}"
    text: "AGATE"
```

```yaml

  - type: FILE
    op: EXEC
    action: READ
    path: "{B[baseDir]}/{B[filename]}"
    response: report

```

Unterstützte Platzhalter folgen den allgemeinen AGATE-Regeln, beispielsweise:

```
{B[var]}
{E[env.*]}
{XL[column]}

```

---

20. Vollständiges Beispiel

```yaml
testCases:

  - id: TC_FILE_FULL_DEMO
    description: "FILE Engine - Full Happy Path"
    stage: "*"
    priority: HIGH

    variables:
      baseDir: "output/file-demo"
      filename: "test.txt"
      copyFilename: "test-copy.txt"
      movedFilename: "test-moved.txt"

    steps:

      # Datei schreiben
      - type: FILE
        op: EXEC
        action: WRITE
        path: "{B[baseDir]}/{B[filename]}"
        text: |
          AGATE FILE Engine
          Line 2
          CHANGED
          Line 4
          CHANGED
        response: write_result

      # Existenz direkt prüfen
      - type: FILE
        op: ASSERT
        action: EXISTS
        path: "{B[baseDir]}/{B[filename]}"

      # Existenz als Boolean speichern
      - type: FILE
        op: EXEC
        action: EXISTS
        path: "{B[baseDir]}/{B[filename]}"
        response: file_exists

      # Datei lesen
      - type: FILE
        op: EXEC
        action: READ
        path: "{B[baseDir]}/{B[filename]}"
        response: file_content

      # Inhalt prüfen
      - type: FILE
        op: ASSERT
```

```
        response: file_content
        action: CONTAINS
        value: "AGATE FILE Engine"

      - type: FILE
        op: ASSERT
        response: file_content
        action: NOT_CONTAINS
        value: "ERROR"

      - type: FILE
        op: ASSERT
        response: file_content
        action: COUNT
        value: "CHANGED"
        expected: 2

      # Vollständigen Inhalt puffern
      - type: FILE
        op: BUFFER
        response: file_content
        action: TEXT
        name: complete_file

      # Gefilterte Zeilen puffern
      - type: FILE
        op: BUFFER
        response: file_content
        action: FILTER
        value: "CHANGED"
        name: changed_lines

      # Erste Zeile
      - type: FILE
        op: BUFFER
        response: file_content
        action: LINE
        value: 0
        name: first_line

      # Letzte Zeile
      - type: FILE
        op: BUFFER
        response: file_content
        action: LAST_LINE
        value: 0
        name: last_line

      # Anzahl
      - type: FILE
        op: BUFFER
        response: file_content
        action: COUNT
        value: "CHANGED"
        name: changed_count

      # Text anhängen
      - type: FILE
        op: EXEC
        action: APPEND
        path: "{B[baseDir]}/{B[filename]}"
        text: "APPENDED"
        newline: true

      # Neu lesen
      - type: FILE
        op: EXEC
        action: READ
        path: "{B[baseDir]}/{B[filename]}"
        response: file_after_append

      - type: FILE
```

```
        op: ASSERT
        response: file_after_append
        action: CONTAINS
        value: "APPENDED"

      # Kopieren
      - type: FILE
        op: EXEC
        action: COPY
        source: "{B[baseDir]}/{B[filename]}"
        target: "{B[baseDir]}/{B[copyFilename]}"
        overwrite: true

      - type: FILE
        op: ASSERT
        action: EXISTS
        path: "{B[baseDir]}/{B[copyFilename]}"

      # Verschieben
      - type: FILE
        op: EXEC
        action: MOVE
        source: "{B[baseDir]}/{B[copyFilename]}"
        target: "{B[baseDir]}/{B[movedFilename]}"
        overwrite: true

      - type: FILE
        op: ASSERT
        action: NOT_EXISTS
        path: "{B[baseDir]}/{B[copyFilename]}"

      - type: FILE
        op: ASSERT
        action: EXISTS
        path: "{B[baseDir]}/{B[movedFilename]}"

      # Löschen
      - type: FILE
        op: EXEC
        action: DELETE
        path: "{B[baseDir]}/{B[movedFilename]}"

      - type: FILE
        op: ASSERT
        action: NOT_EXISTS
        path: "{B[baseDir]}/{B[movedFilename]}"

      - type: FILE
        op: EXEC
        action: DELETE
        path: "{B[baseDir]}/{B[filename]}"

      # Cleanup darf bereits fehlende Datei akzeptieren
      - type: FILE
        op: EXEC
        action: DELETE
        path: "{B[baseDir]}/{B[filename]}"
        missingOk: true

```

---

21. Troubleshooting & Best Practices

Datei wird nicht gefunden

Prüfe:

- ob der Pfad korrekt ist,

- ob ein relativer Pfad vom erwarteten AGATE-Arbeitsverzeichnis ausgeht,
- ob vorherige Schritte die Datei tatsächlich erzeugt haben.

Bei Unsicherheit kann zunächst geprüft werden:

```
- type: FILE
  op: ASSERT
  action: EXISTS
  path: "output/report.txt"

```

Relative Pfade

Für portable Testfälle sollten relative Pfade bevorzugt werden:

```
path: "output/report.txt"

```

Absolute Pfade wie:

```
path: "C:\\tmp\\report.txt"

```

sind stärker an die konkrete Ausführungsumgebung gekoppelt.

Cleanup

Für Cleanup-Schritte ist häufig sinnvoll:

```
missingOk: true

```

Beispiel:

```
- type: FILE
  op: EXEC
  action: DELETE
  path: "output/temp.txt"
  missingOk: true

```

Dadurch bleibt ein Cleanup idempotent.

BUFFER benötigt READ

`FILE BUFFER` arbeitet auf einem zuvor gespeicherten Dateiinhalt.

Falsch:

```
- type: FILE
  op: BUFFER
  path: "output/report.txt"
  action: TEXT
  name: report

```

Richtig:

```yaml
- type: FILE
  op: EXEC
  action: READ
  path: "output/report.txt"
  response: report_raw

- type: FILE
  op: BUFFER
  response: report_raw
  action: TEXT
  name: report

```

Content ASSERT benötigt response

Auch Content Assertions arbeiten auf einem zuvor gelesenen Inhalt.

```yaml
- type: FILE
  op: EXEC
  action: READ
  path: "output/report.txt"
  response: report

- type: FILE
  op: ASSERT
  response: report
  action: CONTAINS
  value: "SUCCESS"

```

Filesystem Assertions verwenden dagegen direkt `path`:

```yaml
- type: FILE
  op: ASSERT
  action: EXISTS
  path: "output/report.txt"

```

---

22. Kurzreferenz

```
FILE
├── EXEC
│   ├── READ
│   │   ├── path
│   │   ├── response
│   │   └── encoding [default: UTF-8]
│   │
│   ├── WRITE
│   │   ├── path
│   │   ├── text
│   │   ├── encoding [default: UTF-8]
│   │   └── overwrite
│   │
│   ├── APPEND
│   │   ├── path
│   │   ├── text
│   │   ├── encoding [default: UTF-8]
│   │   └── newline [default: false]
│   │
│   ├── COPY
│   │   ├── source
│   │   ├── target
│   │   └── overwrite [default: false]
│   │
│   ├── MOVE
```

```
26 │   │   ├── source
27 │   │   ├── target
28 │   │   └── overwrite [default: false]
29 │   │
30 │   ├── DELETE
31 │   │   ├── path
32 │   │   └── missingOk [default: false]
33 │   │
34 │   └── EXISTS
35 │       ├── path
36 │       └── response
37 │
38 ├── BUFFER
39 │   ├── TEXT
40 │   ├── FILTER
41 │   ├── LINE
42 │   ├── LAST_LINE
43 │   └── COUNT
44 │
45 └── ASSERT
46     ├── EXISTS
47     ├── NOT_EXISTS
48     ├── CONTAINS
49     ├── NOT_CONTAINS
50     ├── EQUALS
51     ├── NOT_EQUALS
52     └── COUNT
53
```

---

23. Design-Grundsatz

Die FILE Engine folgt bewusst demselben AGATE-Muster wie CMD und OC:

```
EXEC
    -> Operation ausführen

BUFFER
    -> Ergebnis extrahieren

ASSERT
    -> Ergebnis validieren

```

Dadurch bleibt die DSL über verschiedene Engines hinweg konsistent.

Ein Benutzer muss nicht für jede technische Domäne ein vollständig neues Bedienkonzept lernen.

Die FILE Engine abstrahiert dabei ausschließlich allgemeine, plattformunabhängige Dateioperationen und bildet keine vollständige Betriebssystem-Shell nach.